

Subsetting Data in R

SISBID: Data Wrangling in R

Subsetting part 2

Recap

We talked about many things yesterday!

- using the GUI or `read_csv` & `read_delim` from the `readr` package to import data (and that `readxl` another package is needed for excel files)
- saving data using `write_csv` & `write_rds`
- that there are many practices that help reproducibility
 - testing your rmarkdown knit regularly
 - cleaning your environment
 - using rproj files
 - using RMarkdown and describing your code and data sources

Recap cont.

- `filter` helps us filter rows based on conditions of columns
- `pull` pulls out the values of a column into a vector
- `select` grabs columns to make a smaller table
- the `naniar` package is really helpful for missing data
- the decision to remove **NA** values or recode them, depends on your data and what the values mean

Back to Subsetting!

Data

Let's continue to work with the UFO dataset.

We will often use the `glimpse()` function of the `dplyr` package of the `tidyverse` to look at a rotated view of the data.

```
library(tidyverse)
ufo <- read_csv(
  "https://sisbid.github.io/Data-Wrangling/data/ufo/ufo_data_complete.csv")
```

```
Warning: One or more parsing issues, call `problems()` on your data frame for
  dat <- vroom(...)
  problems(dat)
```

```
Rows: 88875 Columns: 11
```

```
— Column specification —————
```

```
Delimiter: ","
```

```
chr (10): datetime, city, state, country, shape, duration (hours/min), comme.
```

```
dbl (1): duration (seconds)
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this mess
```

Let's learn more about this data

We might decide to rename some columns to make them easier to work with.

We need backticks to work with special characters like (and spaces.

Renaming Columns of a data frame or tibble

To rename columns in `dplyr`, you can use the `rename` function.

Notice the new name is listed **first**!

```
# general format! not code!
```

```
{data you are creating or changing} <- {data you are using} %>%  
  rename({New Name} = {Old name})
```

```
ufo_2 <- ufo %>%  
  rename(duration_seconds = `duration (seconds)`)  
head(ufo_2, n = 3)
```

```
# A tibble: 3 × 11
```

```
  datetime      city state country shape duration_seconds `duration (hours/min)`  
  <chr>         <chr> <chr> <chr>  <chr>          <dbl> <chr>  
1 10/10/1949 ... san ... tx    us    cyli...      2700 45 minutes  
2 10/10/1949 ... lack... tx    <NA>    light      7200 1-2 hrs  
3 10/10/1955 ... ches... <NA>    gb     circ...       20 20 seconds  
# i 4 more variables: comments <chr>, `date posted` <chr>, latitude <chr>,  
# longitude <chr>
```


More Renaming

```
ufo_2<- ufo %>%  
  rename(duration_seconds = `duration (seconds)`,  
          duration_h_m = `duration (hours/min)`)  
glimpse(ufo_2)
```

Rows: 88,875

Columns: 11

\$ datetime	<chr> "10/10/1949 20:30", "10/10/1949 21:00", "10/10/1955 1
\$ city	<chr> "san marcos", "lackland afb", "chester (uk/england)"
\$ state	<chr> "tx", "tx", NA, "tx", "hi", "tn", NA, "ct", "al", "f
\$ country	<chr> "us", NA, "gb", "us", "us", "us", "gb", "us", "us", "
\$ shape	<chr> "cylinder", "light", "circle", "circle", "light", "sp
\$ duration_seconds	<dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, 30
\$ duration_h_m	<chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", "
\$ comments	<chr> "This event took place in early fall around 1949-50.
\$ `date posted`	<chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004",
\$ latitude	<chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21.4
\$ longitude	<chr> "-97.9411111", "-98.581082", "-2.916667", "-96.645833

Unusual Column Names

It's best to avoid unusual column names where possible, as things get tricky later.

We just showed the use of ``` backticks.

Other atypical column names are those with:

- spaces
- number without characters
- number starting the name
- other punctuation marks (besides "_" or "." and not at the beginning)

A solution!

Rename tricky column names so that you don't have to deal with them later!



Renaming all columns of a data frame: dplyr

To rename all columns you use the `rename_with()`. In this case we will use `toupper()` to make all letters upper case. Could also use `tolower()` function.

```
ufo_upper <- ufo %>% rename_with(toupper)
head(ufo_upper, 2)
```

```
# A tibble: 2 × 11
  DATETIME CITY STATE COUNTRY SHAPE `DURATION (SECONDS)` `DURATION (HOURS/MIN)`
  <chr>    <chr> <chr> <chr>   <chr>          <dbl> <chr>
1 10/10/1... san ... tx    us    cyli...      2700 45 minutes
2 10/10/1... lack... tx    <NA>   light      7200 1-2 hrs
# i 4 more variables: COMMENTS <chr>, `DATE POSTED` <chr>, LATITUDE <chr>,
# LONGITUDE <chr>
```

```
ufo_upper %>% rename_with(tolower) %>% head(n = 2)
```

```
# A tibble: 2 × 11
  datetime city state country shape `duration (seconds)` `duration (hours/min)`
  <chr>    <chr> <chr> <chr>   <chr>          <dbl> <chr>
1 10/10/1... san ... tx    us    cyli...      2700 45 minutes
2 10/10/1... lack... tx    <NA>   light      7200 1-2 hrs
# i 4 more variables: comments <chr>, `date posted` <chr>, latitude <chr>,
# longitude <chr>
```

Janitor package

```
#install.packages("janitor")
library(janitor)
ufo <- clean_names(ufo)
head(ufo)
```

```
# A tibble: 6 × 11
  datetime          city state country shape duration_seconds duration_hours_min
  <chr>            <chr> <chr> <chr>  <chr>          <dbl> <chr>
1 10/10/1949 20:30 san ... tx    us    cyli..        2700 45 minutes
2 10/10/1949 21:00 lack... tx    <NA>    light        7200 1-2 hrs
3 10/10/1955 17:00 ches... <NA> gb     circ...        20 20 seconds
4 10/10/1956 21:00 edna  tx    us     circ...        20 1/2 hour
5 10/10/1960 20:00 kane... hi    us     light         900 15 minutes
6 10/10/1961 19:00 bris... tn    us     sphe...        300 5 minutes
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
# longitude <chr>
```

Subset based on a class

The `where()` function can help select columns of a specific class

`is.character()` and `is.numeric()` are often the most helpful

```
head(ufo, 2)
```

```
# A tibble: 2 × 11
  datetime          city state country shape duration_seconds duration_hours_r
  <chr>            <chr> <chr> <chr>   <chr>          <dbl> <chr>
1 10/10/1949 20:30 san ... tx      us      cyli...      2700 45 minutes
2 10/10/1949 21:00 lack... tx      <NA>      light      7200 1-2 hrs
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
#   longitude <chr>
```

```
ufo %>% select(where(is.numeric)) %>% head(n = 2)
```

```
# A tibble: 2 × 1
  duration_seconds
  <dbl>
1          2700
2          7200
```

Adding/Removing Columns

Adding columns to a data frame: dplyr (tidyverse way)

The `mutate` function in `dplyr` allows you to add or modify columns of a data frame.

General format – Not the code!

```
{data object to update} <- {data to use} %>%  
  mutate({new variable name} = {new variable source})
```

```
ufo %>%  
  mutate(state_upper = toupper(state)) %>% glimpse()
```

Rows: 88,875

Columns: 12

```
$ datetime      <chr> "10/10/1949 20:30", "10/10/1949 21:00", "10/10/1955...  
$ city          <chr> "san marcos", "lackland afb", "chester (uk/england)...  
$ state         <chr> "tx", "tx", NA, "tx", "hi", "tn", NA, "ct", "al", "...  
$ country       <chr> "us", NA, "gb", "us", "us", "us", "gb", "us", "us",...  
$ shape         <chr> "cylinder", "light", "circle", "circle", "light", "...  
$ duration_seconds <dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, ...  
$ duration_hours_min <chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", ...  
$ comments      <chr> "This event took place in early fall around 1949-50...  
$ date_posted   <chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004"...  
$ latitude      <chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21...  
$ longitude     <chr> "-97.9411111", "-98.581082", "-2.916667", "-96.6458...  
$ state_upper   <chr> "TX", "TX", NA, "TX", "HI", "TN", NA, "CT", "AL", "...
```

Use mutate to modify existing columns

The `mutate` function in `dplyr` allows you to add or modify columns of a data frame.

General format – Not the code!

```
{data object to update} <- {data to use} %>%  
  mutate({variable name to change} = {variable modification})
```

```
ufo %>%  
  mutate(state = toupper(state)) %>% glimpse()
```

```
Rows: 88,875  
Columns: 11  
$ datetime      <chr> "10/10/1949 20:30", "10/10/1949 21:00", "10/10/1955...  
$ city          <chr> "san marcos", "lackland afb", "chester (uk/england)...  
$ state         <chr> "TX", "TX", NA, "TX", "HI", "TN", NA, "CT", "AL", "...  
$ country       <chr> "us", NA, "gb", "us", "us", "us", "gb", "us", "us",...  
$ shape         <chr> "cylinder", "light", "circle", "circle", "light", "...  
$ duration_seconds <dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, ...  
$ duration_hours_min <chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", ...  
$ comments      <chr> "This event took place in early fall around 1949-50...  
$ date_posted   <chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004"...  
$ latitude      <chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21...  
$ longitude     <chr> "-97.9411111", "-98.581082", "-2.916667", "-96.6458...
```

remember to save your data

If you want to actually make the change you need to reassign the data object.

```
ufo <- ufo %>%  
  mutate(state = toupper(state))
```

Removing columns of a data frame: dplyr

The `select` function can remove a column with minus (`-`)

```
select(ufo, - datetime) %>% glimpse()
```

Rows: 88,875

Columns: 10

```
$ city      <chr> "san marcos", "lackland afb", "chester (uk/england)..."
$ state     <chr> "tx", "tx", NA, "tx", "hi", "tn", NA, "ct", "al", "...
$ country   <chr> "us", NA, "gb", "us", "us", "us", "gb", "us", "us",...
$ shape     <chr> "cylinder", "light", "circle", "circle", "light", "...
$ duration_seconds <dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, ...
$ duration_hours_min <chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", ...
$ comments  <chr> "This event took place in early fall around 1949-50..."
$ date_posted <chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004"...
$ latitude  <chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21..."
$ longitude <chr> "-97.9411111", "-98.581082", "-2.916667", "-96.6458..."
```

Or, you can simply select the columns you want to keep, ignoring the ones you want to remove.

Removing columns in a data frame: dplyr

You can use `c()` to list the columns to remove or tidyhelpers.

```
select(ufo, -(starts_with("c"))) %>% glimpse()
```

Rows: 88,875

Columns: 8

```
$ datetime      <chr> "10/10/1949 20:30", "10/10/1949 21:00", "10/10/1955...
$ state         <chr> "tx", "tx", NA, "tx", "hi", "tn", NA, "ct", "al", "...
$ shape         <chr> "cylinder", "light", "circle", "circle", "light", "...
$ duration_seconds <dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, ...
$ duration_hours_min <chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", ...
$ date_posted    <chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004"...
$ latitude       <chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21...
$ longitude      <chr> "-97.9411111", "-98.581082", "-2.916667", "-96.6458..."
```

Ordering columns

Ordering the columns of a data frame: dplyr

The `select` function can reorder columns.

```
head(ufo, n = 2)
```

```
# A tibble: 2 × 11
  datetime          city state country shape duration_seconds duration_hours_r
  <chr>             <chr> <chr> <chr>   <chr>         <dbl> <chr>
1 10/10/1949 20:30 san ... tx      us      cyli...      2700 45 minutes
2 10/10/1949 21:00 lack... tx      <NA>      light      7200 1-2 hrs
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
#   longitude <chr>
```

```
ufo %>% select(country, shape, datetime) %>% head(n = 2)
```

```
# A tibble: 2 × 3
  country shape      datetime
  <chr>   <chr>   <chr>
1 us     cylinder 10/10/1949 20:30
2 <NA>    light    10/10/1949 21:00
```

Ordering the columns of a data frame: dplyr

In addition to `select` we can also use the `relocate()` function of dplyr to rearrange the columns for more complicated moves.

```
head(ufo, n = 2)
```

```
# A tibble: 2 × 11
  datetime      city state country shape duration_seconds duration_hours_min
  <chr>         <chr> <chr> <chr>  <chr>          <dbl> <chr>
1 10/10/1949 20:30 san ... tx    us    cylin...    2700 45 minutes
2 10/10/1949 21:00 lack... tx    <NA>    light    7200 1-2 hrs
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
#   longitude <chr>
```

```
ufo %>% relocate(datetime, .after = shape) %>% head(n = 2)
```

```
# A tibble: 2 × 11
  city      state country shape datetime duration_seconds duration_hours_min
  <chr>     <chr> <chr>  <chr>  <chr>          <dbl> <chr>
1 san marcos tx    us    cylin... 10/10/1...    2700 45 minutes
2 lackland afb tx    <NA>    light 10/10/1...    7200 1-2 hrs
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
#   longitude <chr>
```


Ordering the columns of a data frame: dplyr

In addition to `select` we can also use the `relocate()` function of dplyr to rearrange the columns for more complicated moves.

```
head(ufo, n = 2)
```

```
# A tibble: 2 × 11
  datetime          city state country shape duration_seconds duration_hours_min
  <chr>             <chr> <chr> <chr>  <chr>          <dbl> <chr>
1 10/10/1949 20:30 san ... tx    us    cyli...      2700 45 minutes
2 10/10/1949 21:00 lack... tx    <NA>    light      7200 1-2 hrs
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
#   longitude <chr>
```

```
ufo %>% relocate(shape, .before = city) %>% head(n = 2)
```

```
# A tibble: 2 × 11
  datetime          shape city state country duration_seconds duration_hours_min
  <chr>             <chr> <chr> <chr> <chr>          <dbl> <chr>
1 10/10/1949 20:30 cyli... san ... tx    us      2700 45 minutes
2 10/10/1949 21:00 light lack... tx    <NA>      7200 1-2 hrs
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
#   longitude <chr>
```

Ordering rows

Ordering the rows of a data frame: dplyr

The `arrange` function can reorder rows By default, `arrange` orders in increasing order:

```
ufo %>% arrange(duration_seconds)
```

```
# A tibble: 88,875 × 11
```

	datetime <chr>		city <chr>	state <chr>	country <chr>	shape <chr>	duration_seconds <dbl>	duration_hours_r <chr>
1	10/10/1995	17:...	puer...	pr	<NA>	<NA>	0	<NA>
2	10/10/1999	21:...	ashl...	mo	us	light	0	two seperate tir
3	10/10/2002	22:...	baha...	<NA>	<NA>	egg	0	<NA>
4	10/10/2002	22:...	burn...	<NA>	au	cross	0	12
5	10/10/2005	11:...	edge...	fl	us	<NA>	0	300
6	10/10/2005	24:...	fran...	in	us	disk	0	?
7	10/10/2006	23:...	knik	ak	us	tria...	0	5
8	10/10/2007	05:...	bake...	ca	us	circ...	0	had a call of a
9	10/10/2008	09:...	amar...	tx	us	flash	0	<NA>
10	10/10/2009	00:...	gree...	<NA>	<NA>	rect...	0	<NA>

```
# i 88,865 more rows
```

```
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
```

```
# longitude <chr>
```

Ordering the rows of a data frame: dplyr

Use the `desc` to arrange the rows in descending order:

```
ufo %>% arrange(desc(duration_seconds))
```

```
# A tibble: 88,875 × 11
  datetime      city state country shape duration_seconds duration_hours_r
  <chr>         <chr> <chr> <chr>   <chr>         <dbl> <chr>
1 10/1/1983 17:00 birm... <NA>   gb      sphe...     97836000 31 years
2 6/3/2010 23:30 otta... on      ca      other     82800000 23000hrs
3 9/15/1991 18:00 gree... ar      us      light     66276000 21 years
4 4/2/1983 24:00 dont... <NA>   <NA>    <NA>     52623200 2 months
5 8/10/2012 21:00 finl... wa      us      light     52623200 2 months
6 8/24/2002 01:00 engl... fl      us      light     52623200 2 months
7 6/30/1969 22:45 some... <NA>   gb      cone      25248000 8 years
8 10/7/2013 20:00 okla... ok      <NA>    circ...    10526400 4 months
9 3/1/1994 01:00 meni... ca      us      unkn...    10526400 4 months
10 8/3/2008 21:00 virg... va      us      fire...    10526400 4 months
# i 88,865 more rows
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
# longitude <chr>
```

Ordering the rows of a data frame: dplyr

You can combine increasing and decreasing orderings. The first listed gets priority.

```
arrange(ufo, desc(duration_seconds), shape)
```

```
# A tibble: 88,875 × 11
  datetime      city state country shape duration_seconds duration_hours_r
  <chr>         <chr> <chr> <chr>   <chr>      <dbl> <chr>
1 10/1/1983 17:00 birm... <NA>   gb      sphe...  97836000 31 years
2 6/3/2010 23:30 otta... on     ca      other  82800000 23000hrs
3 9/15/1991 18:00 gree... ar     us      light  66276000 21 years
4 8/10/2012 21:00 finl... wa     us      light  52623200 2 months
5 8/24/2002 01:00 engl... fl     us      light  52623200 2 months
6 4/2/1983 24:00 dont... <NA>   <NA>   <NA>   52623200 2 months
7 6/30/1969 22:45 some... <NA>   gb      cone   25248000 8 years
8 10/7/2013 20:00 okla... ok     <NA>   circ... 10526400 4 months
9 8/3/2008 21:00 virg... va     us      fire... 10526400 4 months
10 3/1/1994 01:00 meni... ca     us      unkn... 10526400 4 months
# i 88,865 more rows
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
# longitude <chr>
```

Ordering the rows of a data frame: dplyr

You can combine increasing and decreasing orderings. The first listed gets priority.

```
arrange(ufo, shape, desc(duration_seconds))
```

```
# A tibble: 88,875 × 11
  datetime      city state country shape duration_seconds duration_hours_r
  <chr>         <chr> <chr> <chr>   <chr>      <dbl> <chr>
1 6/24/1996 00:30 auro... co      us      chan...      3600 1 hour
2 12/1/2007 12:00 kail... hi      us      chan...    172800 22 days ongoing
3 6/25/2011 09:30 geor... tx      us      chan...    172800 days
4 11/27/2002 04:... san ... ca      us      chan...    109800 hour
5 8/16/1968 03:00 kans... ks      <NA>      chan...     97200 27 hours
6 2/25/2006 06:00 cano... ca      us      chan...     86400 off/on ti&#39;l 2
7 5/1/2008 04:30 kans... mo      us      chan...     73800 2 1/2 hours
8 7/16/2008 23:00 clar... wa      us      chan...     73800 at least 2 1/2 l
9 3/15/2000 06:39 wint... ma      us      chan...     50400 14 hours
10 9/1/1994 11:00 gran... or      us      chan...     43200 12 hours
# i 88,865 more rows
# i 4 more variables: comments <chr>, date_posted <chr>, latitude <chr>,
# longitude <chr>
```

Summary

- `rename` can change a name - new name = old name
- `clean_names` of the `janitor` package can change many names
- `select()` and `relocate()` can be used to reorder columns
- can remove a column in a few ways:
 - using `select()` with negative sign in front of column name(s)
 - just not selecting it
- `mutate()` can be used to modify an existing variable or make a new variable
- `arrange()` can be used to reorder rows
- can arrange in descending order with `desc()`

Lab

[Link to Lab](#)